

ASSIGNMENT-6.4

2303A52249

A. VAMSHI KRISHNA

Task 1: Student Performance Evaluation System

Scenario: You are building a simple academic management module for a university system.

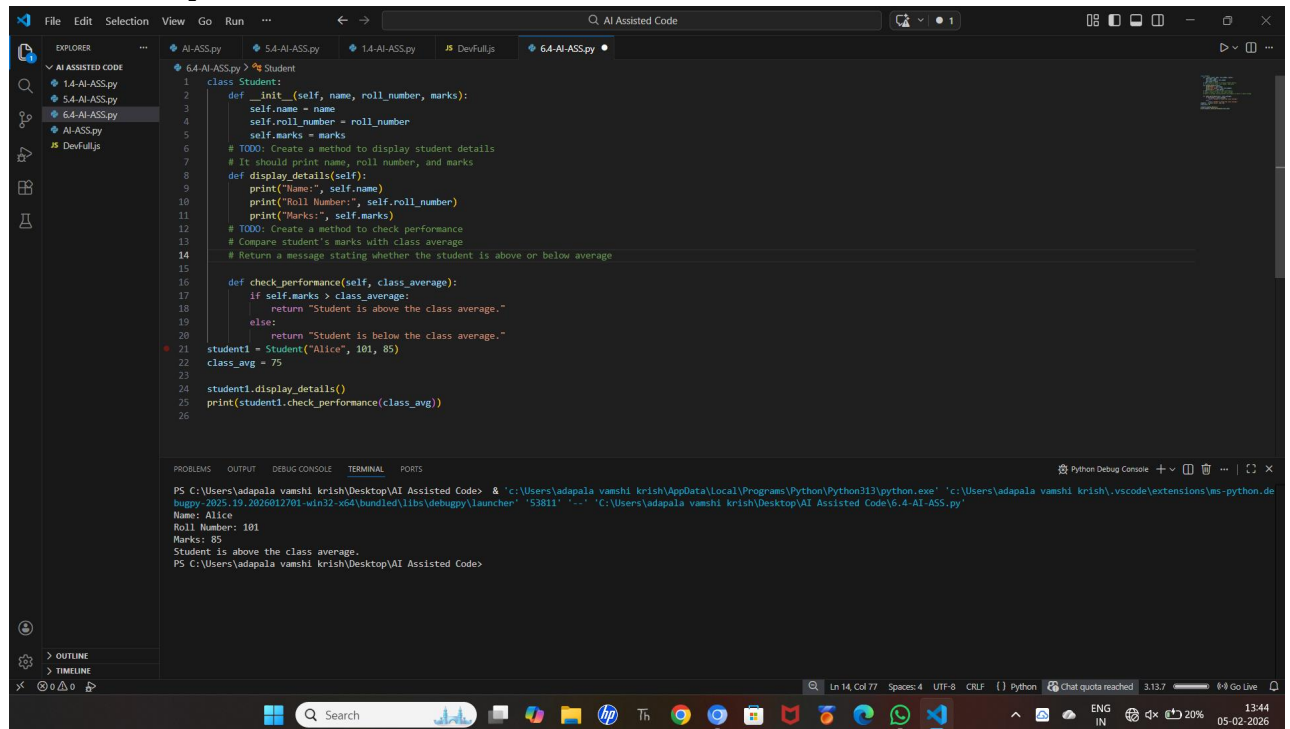
Prompt:

Create a Python class named Student with attributes name, roll_number, and marks.

Using GitHub Copilot, generate:

1. A method to display student details
 2. A method to check whether the student's marks are above the class average
- Use conditional statements and self attributes.

Code and Output:



```
1 class Student:
2     def __init__(self, name, roll_number, marks):
3         self.name = name
4         self.roll_number = roll_number
5         self.marks = marks
6         # TODO: Create a method to display student details
7         # It should print name, roll number, and marks
8     def display_details(self):
9         print("Name:", self.name)
10        print("Roll Number:", self.roll_number)
11        print("Marks:", self.marks)
12        # TODO: Create a method to check performance
13        # Compare student's marks with class average
14        # Return a message stating whether the student is above or below average
15
16    def check_performance(self, class_average):
17        if self.marks > class_average:
18            return "Student is above the class average."
19        else:
20            return "Student is below the class average."
21
22 student1 = Student("Alice", 101, 85)
23 class_avg = 75
24 student1.display_details()
25 print(student1.check_performance(class_avg))
26
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\adapala vamsi krish\Desktop\AI Assisted Code> & 'c:\Users\adapala vamsi krish\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\adapala vamsi krish\.vscode\extensions\ms-python.de
bugpy-2025.19.2026012701-win32-x64\bundled\libs\debugpy\launcher' '53811' '-' 'c:\Users\adapala vamsi krish\Desktop\AI Assisted Code\6.4-AI-ASS.py'
Name: Alice
Roll Number: 101
Marks: 85
Student is above the class average.
PS C:\Users\adapala vamsi krish\Desktop\AI Assisted Code>
```

Observation:

GitHub Copilot successfully generated methods using self attributes.

The display_details() method prints student information clearly.

The check_performance() method correctly uses an if-else condition to compare marks with the class average.

The output correctly indicates whether the student is above or below the class average

Task 2: Data Processing in a Monitoring System

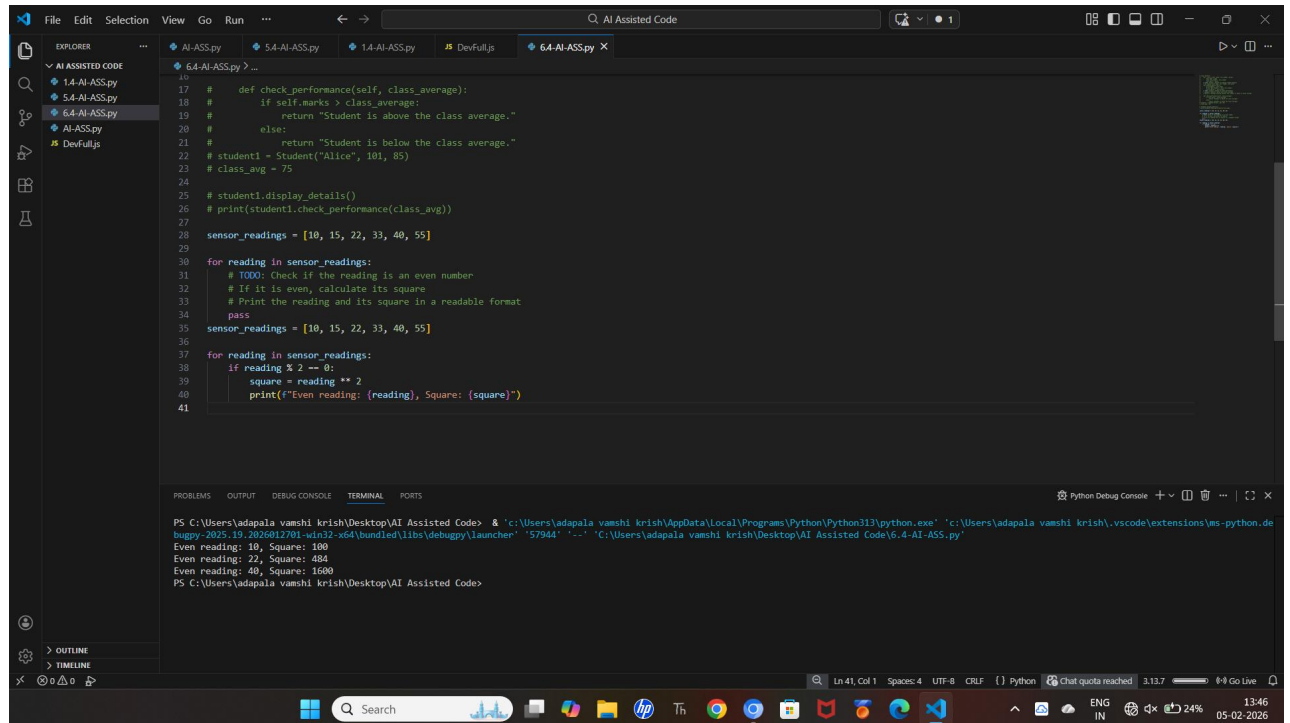
Scenario:

You are working on a basic data monitoring script where sensor readings are collected as numbers. Only even readings need further processing.

Prompt: Iterate over a list of sensor readings. Using GitHub Copilot:

- Identify even numbers
 - Calculate the square of even numbers
 - Print the result in a readable format
- Use modulus operator and conditional statements.

Code and Output:



The screenshot shows a Visual Studio Code editor window with a Python file named `6.4-AI-ASS.py`. The code defines a `Student` class with a `check_performance` method and a list of sensor readings. It then iterates over the readings, identifying even numbers and calculating their squares. The output is printed in a readable format.

```
16
17 # def check_performance(self, class_average):
18 #     if self.marks > class_average:
19 #         return "Student is above the class average."
20 #     else:
21 #         return "Student is below the class average."
22 # student1 = Student("Alice", 101, 85)
23 # class_avg = 75
24
25 # student1.display_details()
26 # print(student1.check_performance(class_avg))
27
28 sensor_readings = [10, 15, 22, 33, 40, 55]
29
30 for reading in sensor_readings:
31     # TODO: Check if the reading is an even number
32     # If it is even, calculate its square
33     # Print the reading and its square in a readable format
34     pass
35 sensor_readings = [10, 15, 22, 33, 40, 55]
36
37 for reading in sensor_readings:
38     if reading % 2 == 0:
39         square = reading ** 2
40         print(f"Even reading: {reading}, Square: {square}")
41
```

The terminal output shows the execution of the script, displaying the squares of the even readings:

```
PS C:\Users\adapala vamshi krish\Desktop\AI Assisted Code> python .\6.4-AI-ASS.py
Even reading: 10, Square: 100
Even reading: 22, Square: 484
Even reading: 40, Square: 1600
PS C:\Users\adapala vamshi krish\Desktop\AI Assisted Code>
```

Observation:

Copilot correctly identified even numbers using the modulus (%) operator. The square of each even reading was calculated accurately. Conditional statements were used effectively to filter even numbers. Output was printed in a clear and readable format.

Task 3: Banking Transaction Simulation

Scenario:

You are developing a basic banking module that handles deposits and withdrawals for customers.

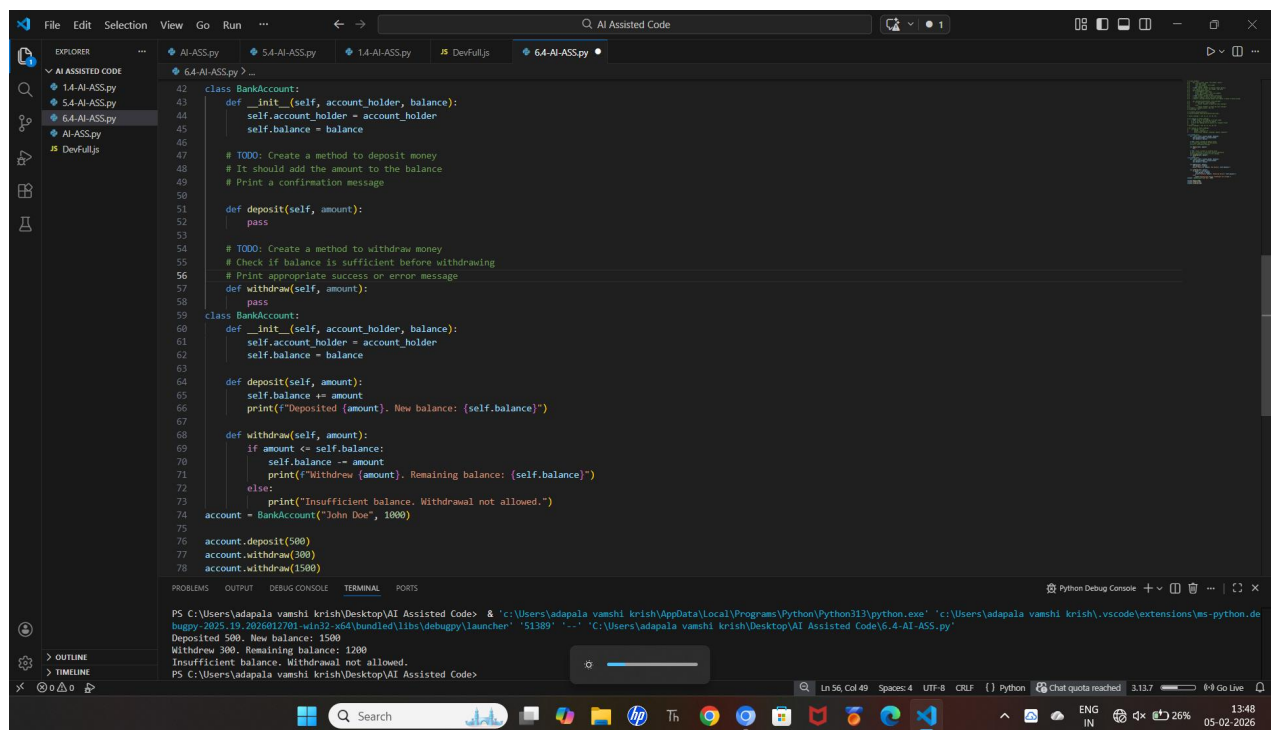
Prompt:

Create a Python class BankAccount with attributes account_holder and balance.

Using GitHub Copilot, generate methods to:

- Deposit money
 - Withdraw money
 - Prevent withdrawal if balance is insufficient
- Use if-else conditions and self attributes.

Code and Output:



```
42 class BankAccount:
43     def __init__(self, account_holder, balance):
44         self.account_holder = account_holder
45         self.balance = balance
46
47     # TODO: Create a method to deposit money
48     # It should add the amount to the balance
49     # Print a confirmation message
50
51     def deposit(self, amount):
52         pass
53
54     # TODO: Create a method to withdraw money
55     # Check if balance is sufficient before withdrawing
56     # Print appropriate success or error message
57
58     def withdraw(self, amount):
59         pass
60
61 class BankAccount:
62     def __init__(self, account_holder, balance):
63         self.account_holder = account_holder
64         self.balance = balance
65
66     def deposit(self, amount):
67         self.balance += amount
68         print(f"Deposited {amount}. New balance: {self.balance}")
69
70     def withdraw(self, amount):
71         if amount <= self.balance:
72             self.balance -= amount
73             print(f"Withdrew {amount}. Remaining balance: {self.balance}")
74         else:
75             print("Insufficient balance. Withdrawal not allowed.")
76
77 account = BankAccount("John Doe", 1000)
78
79 account.deposit(500)
80 account.withdraw(300)
81 account.withdraw(1500)
```

```
PS C:\Users\adapala vamshi krish\Desktop\AI Assisted Code> & 'c:\Users\adapala vamshi krish\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\adapala vamshi krish\.vscode\extensions\ms-python.de
bugpy-2025.19.2025012701-win32-x64\bundled\libs\debugpy\launcher' '51389' '-.' 'C:\Users\adapala vamshi krish\Desktop\AI Assisted Code\6.4-AI-ASS.py'
Deposited 500. New balance: 1500
Withdrew 300. Remaining balance: 1200
Insufficient balance. Withdrawal not allowed.
PS C:\Users\adapala vamshi krish\Desktop\AI Assisted Code>
```

Observation:

- Copilot generated functional deposit and withdrawal methods.
- The withdrawal method correctly checks for insufficient balance using if-else.
- Class attributes were accessed properly using self.
- User-friendly messages were displayed for successful and failed transactions.

Task 4: Student Scholarship Eligibility Check

Scenario:

A university wants to identify students eligible for a merit-based scholarship based on their scores.

Prompt:

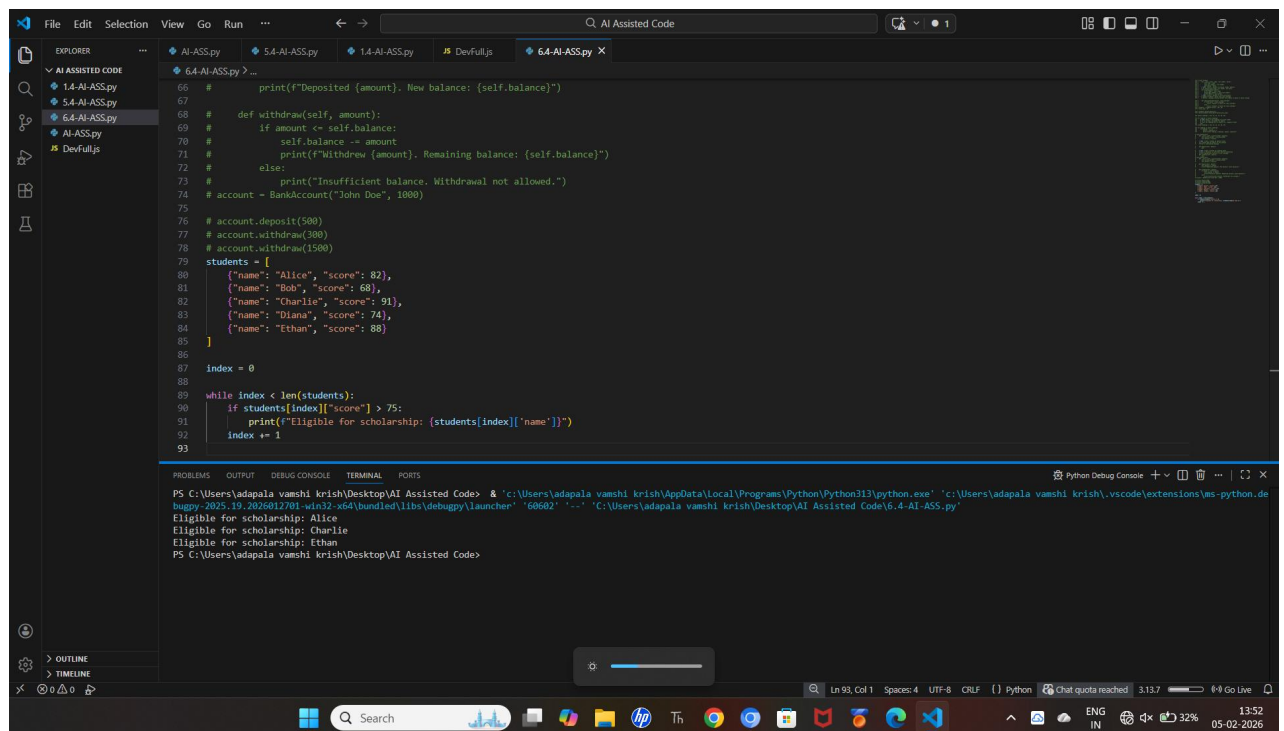
Create a list of dictionaries containing student name and score.

Using GitHub Copilot, generate a while loop that:

- Iterates through the list
- Prints names of students who scored more than 75

Ensure correct index handling.

Code and Output:



```
66 # print(f"Deposited {amount}. New balance: {self.balance}")
67
68 # def withdraw(self, amount):
69 #     if amount <= self.balance:
70 #         self.balance -= amount
71 #         print(f"Withdrew {amount}. Remaining balance: {self.balance}")
72 #     else:
73 #         print("Insufficient balance. Withdrawal not allowed.")
74 # account = BankAccount("John Doe", 1000)
75
76 # account.deposit(500)
77 # account.withdraw(300)
78 # account.withdraw(1500)
79
80 students = [
81     {"name": "Alice", "score": 82},
82     {"name": "Bob", "score": 68},
83     {"name": "Charlie", "score": 91},
84     {"name": "Diana", "score": 74},
85     {"name": "Ethan", "score": 88}
86 ]
87
88 index = 0
89 while index < len(students):
90     if students[index]["score"] > 75:
91         print(f"Eligible for scholarship: {students[index]['name']}")
92     index += 1
93
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Python Debug Console

```
PS C:\Users\adapala vamshi krish\Desktop\AI Assisted Code> & 'c:\Users\adapala vamshi krish\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\adapala vamshi krish\.vscode\extensions\ms-python.de
bugger>2025.19.2026012701-win32-x64\bundled\libs\debugpy\launcher' '68602' '-' 'C:\Users\adapala vamshi krish\Desktop\AI Assisted Code\6.4-AI-ASS.py'
Eligible for scholarship: Alice
Eligible for scholarship: Charlie
Eligible for scholarship: Ethan
PS C:\Users\adapala vamshi krish\Desktop\AI Assisted Code>
```

Ln 93, Col 1 Spaces: 4 UTF-8 CRLF Python Chat quota reached 3.13.7 Go Live

13:52 05-02-2026

Observation:

- Copilot correctly generated a while loop with proper index control.
- The condition to check scores greater than 75 worked as expected.
- Only eligible students were printed.
- The loop executed without index errors.

Task 5: Online Shopping Cart Module

Scenario:

You are designing a simplified shopping cart system for an e-commerce website that supports item management and discount calculation.

Prompt:

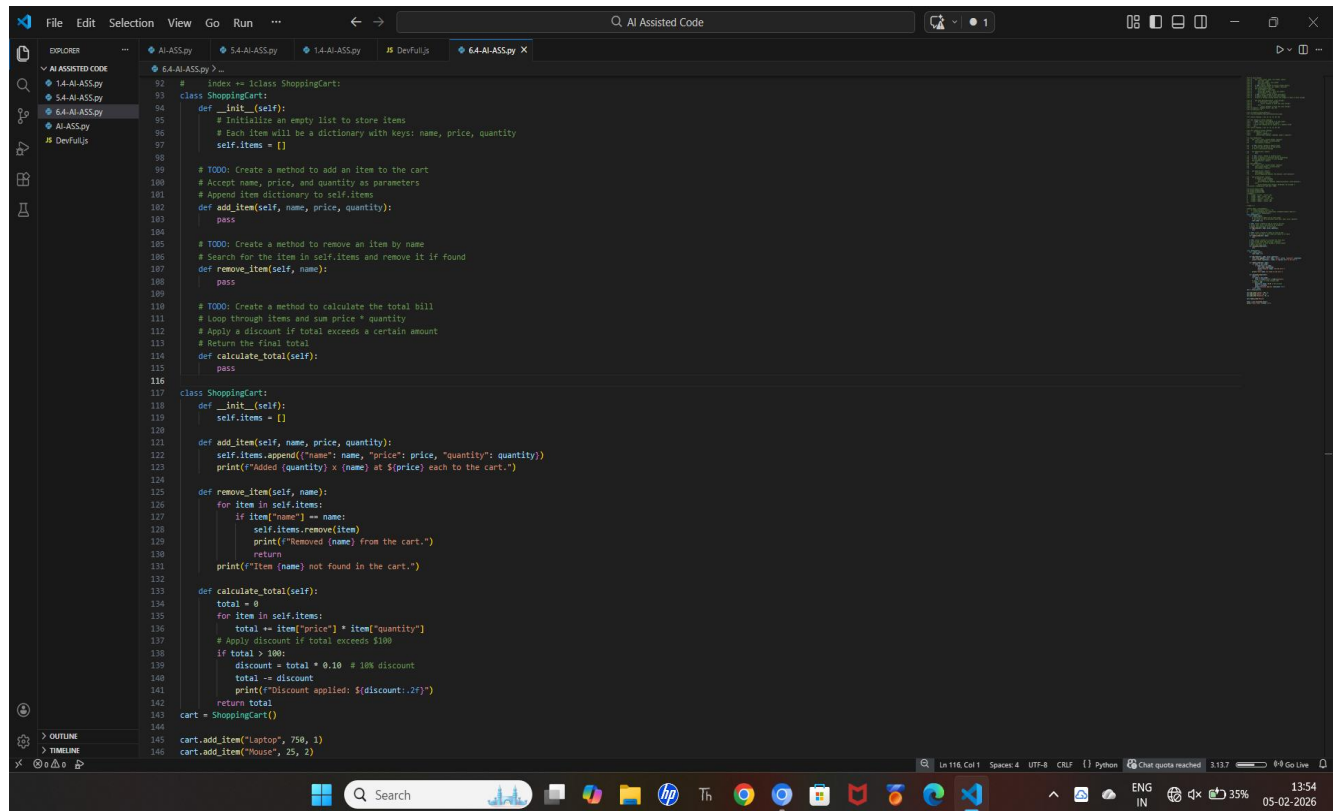
Create a Python class ShoppingCart with an empty list to store items.

Using GitHub Copilot, generate methods to do:

- Add items to the cart
- Remove items from the cart
- Calculate the total bill using a loop
- Apply discount if total exceeds a certain amount

Use loops and conditional logic.

Code and Output:



```
102 # Index -- Class ShoppingCart:
103 class ShoppingCart:
104     def __init__(self):
105         # Initialize an empty list to store items
106         # Each item will be a dictionary with keys: name, price, quantity
107         self.items = []
108
109     # TODO: Create a method to add an item to the cart
110     # Accept name, price, and quantity as parameters
111     # Append item dictionary to self.items
112     def add_item(self, name, price, quantity):
113         pass
114
115     # TODO: Create a method to remove an item by name
116     # Search for the item in self.items and remove it if found
117     def remove_item(self, name):
118         pass
119
120     # TODO: Create a method to calculate the total bill
121     # Loop through items and sum price * quantity
122     # Apply a discount if total exceeds a certain amount
123     # Return the final total
124     def calculate_total(self):
125         pass
126
127 class ShoppingCart:
128     def __init__(self):
129         self.items = []
130
131     def add_item(self, name, price, quantity):
132         self.items.append({"name": name, "price": price, "quantity": quantity})
133         print(f"Added (quantity) x (name) at $(price) each to the cart.")
134
135     def remove_item(self, name):
136         for item in self.items:
137             if item["name"] == name:
138                 self.items.remove(item)
139                 print(f"Removed (name) from the cart.")
140                 return
141         print(f"Item (name) not found in the cart.")
142
143     def calculate_total(self):
144         total = 0
145         for item in self.items:
146             total += item["price"] * item["quantity"]
147         # Apply discount if total exceeds 100
148         if total > 100:
149             discount = total * 0.10 # 10% discount
150             total -= discount
151             print(f"Discount applied: ${discount:.2f}")
152         return total
153
154 cart = ShoppingCart()
155
156 cart.add_item("Laptop", 750, 1)
157 cart.add_item("Mouse", 25, 2)
```

The screenshot shows a VS Code editor with a Python file named `6.4-AI-ASS.py`. The code defines a `ShoppingCart` class with the following methods:

- `add_item(self, name, price, quantity)`: Adds an item to the cart.
- `remove_item(self, name)`: Removes an item from the cart.
- `calculate_total(self)`: Calculates the total bill, applying a 10% discount if the total exceeds \$100.

The terminal output shows the execution of the script, demonstrating the functionality of the shopping cart:

```
\adapala vamshi krish\AppData\Local\Programs\Python\Python313\python.exe "c:\Users\adapala vamshi krish\.vscode\extensions\ms-python.debugpy-2025.19.2026012701-win32-x64\bundled\libs\debugpy\launcher" "56266"
1a vamshi krish\Desktop\AI Assisted Code\6.4-AI-ASS.py
Added 1 x Laptop at $750 each to the cart.
Added 2 x Mouse at $25 each to the cart.
Added 1 x Keyboard at $45 each to the cart.
Removed Mouse from the cart.
Discount applied: $79.50
PS C:\Users\adapala vamshi krish\Desktop\AI Assisted Code> & "c:\Users\adapala vamshi krish\AppData\Local\Programs\Python\Python313\python.exe" "c:\Users\adapala vamshi krish\.vscode\extensions\ms-python.debugpy-2025.19.2026012701-win32-x64\bundled\libs\debugpy\launcher" "56266" "..." "c:\Users\adapala vamshi krish\Desktop\AI Assisted Code\6.4-AI-ASS.py"
Added 1 x Laptop at $750 each to the cart.
Added 2 x Mouse at $25 each to the cart.
Added 1 x Keyboard at $45 each to the cart.
Removed Mouse from the cart.
Discount applied: $79.50
Removed Mouse from the cart.
Discount applied: $79.50
Final total: $715.50
PS C:\Users\adapala vamshi krish\Desktop\AI Assisted Code>
```

Observation:

- Copilot generated all required shopping cart methods successfully.
- Items were added and removed correctly from the cart list.
- The total bill was calculated using a loop.
- Conditional discount logic was applied correctly when the total exceeded the limit.
- The final output reflected accurate billing with discount.